非遗智汇 · 河南非遗智能数字展厅

技术文档

意图控制 · 架构设计 · 可复现性论证

全国高校 Vibe Coding 开发挑战赛 · B 组

选题方向 #5「AI 驱动的高交互教育与可视化」

作者：李宇航 | 仓库：github.com/linchenan666/feiyi-zhihui

数据截至：2026-10-26

摘要

本文档说明《非遗智汇》在 Vibe Coding 模式下的工程实践：人如何通过 Prompt 约束 AI 产出可维护的代码，架构上如何做到模块化源码与单文件编译双轨、离线优先与双层 AI 并存，以及如何用细粒度 Git 历史、构建脚本与演进记录证明作品不是一次性堆砌。

截至 2026-10-26，仓库共 100 次细粒度提交，首末提交跨 2026-09-09 至 2026-10-26 约 7 周；收录 25 项河南国家级非遗、56 张入库图片、10 段讲解音频；构建脚本一键产出 39.01 MB 单文件 HTML，断网功能完整。

第一章　意图控制策略

——人如何驾驭 AI 写代码，而不是被 AI 带着跑

1.1　上下文先于提问

每次向 AI 提问前，先在对话开头给三件事：项目是什么、数据在哪里、硬约束是什么。AI 不靠猜，上下文给足了，输出才不会跑偏。

具体做法：

- 项目定位：一句话说清"纯本地、零后端、单文件、断网可跑"，AI 就不会引入 CDN 依赖。
- 数据位置：告诉 AI 数据在 `data.js` 的 `FEIYI_DATA` 数组里，新增项沿用现有字段结构（name/category/place/region/level/points/deep/qa），AI 就不会自创字段。
- 硬约束：设计系统是宣纸/墨/朱砂/金 60/30/10 配色，字体是子集化内嵌 woff2，hash 路由多视图——这些写进 Prompt，AI 按约束产出，不自由发挥。

> 反例：早期某轮 Prompt 只说"首页要好看"，AI 产出了全站 fade-in-up 动效 + 外部字体 CDN，观感廉价且断网降级。修复方式是下一轮明确"动效只在首页一处入场编排，内容区按需揭示，字体全部内嵌"。

1.2　用 Prompt 锁死架构与代码风格

把架构决策写成 Prompt 约束，让 AI 在每一轮都遵守，而不是每次重新解释。

约束项	Prompt 写法	效果
数据与视图分离	"数据层只写数据，不操作 DOM；视图层只渲染，不写业务逻辑"	data.js 纯数据，app.js 纯逻辑，边界清晰
零 CDN	"不允许引入任何外部 script/link 标签，所有依赖本地 vendor/"	three.js、OrbitControls、Lenis 全部本地化
哈希路由	"用 hash 路由，#/home、#/list、#/detail 等视图切换"	10 视图无刷新切换，无框架
XSS 防护	"所有用户输入和数据渲染前过 esc() 函数"	收藏文案、分享文案均转义
样式规范	"CSS 变量统一在 :root，主色 #8b0000 朱砂、辅色 #2c2c2c 墨、点缀 #d4a017 金"	全项目配色一致，无随机色值

这些约束不是一次写在某个文档里，而是每轮 Prompt 都带一句"保持现有架构和配色，不要引入新依赖"。AI 每次在同一份代码上下文里工作，约束持续生效。

1.3 拆需求喂 AI：一轮一功能

从 evolution-data.js 记录的早期 12 轮迭代看，每轮只做一个模块，不一次性塞全部需求。模块边界清晰，AI 才能聚焦产出可用代码。

轮次	模块	Prompt 核心指令
01	需求解构与骨架	"按赛道要求，把网页改进成终版，必须全程离线可运行"
02	首页视觉	"首页动态文化背景、水墨纸纹质感、可交互导航"
03	展品总览	"每件展品一个专属世界：豫剧=朱砂戏台、钧瓷=窑火金褐"
04	非遗地图	"把地图换成逼真河南地图，加可放大按键，悬停聚焦"
05	城市介绍	"大背景换成城市标志性建筑动态轮播，卡片小字白、大标题黄"
06	知识图谱	"图谱放大时直接在图谱框中央放大，加滚轮缩放"
07	问·AI 双层	"AI 回答流式逐字显现、思考过程可视化；离线无网也能回答"
08	文创生成	"九宫格智能文创生成，生成结果可收藏，加复制分享文案"
09	沉浸演绎	"把沉浸演绎里的播放图片换成权威视频，离线降级静态海报"
10	3D 珍品馆	"3D 建模太粗糙，要按实物一比一完全复刻、里面也是实心的"
11	性能优化	"页面滚轮很卡，加鼠标动态粒子，不要环绕改色"
12	合规交付	"严格按参赛要求，三重思维检查：技术可行性、内容深度、工程合规"

这 12 轮是早期核心功能的迭代记录。全项目从 2026-09-09 至 2026-10-26 共 100 次提交，后续还包括数据扩充（15 项→25 项）、图片版权合规、字体子集化、three.js 本地化、性能修复等多轮打磨。

1.4 引导修复与验证闭环

AI 产出后人工审查、指出问题、让 AI 修复、再验证。每轮记录"Prompt → AI 建议 → 引导修复 → 验证"四步，形成可追溯的演进证据链。

典型修复案例：

- 动效泛滥：AI 首轮全站 fade-in-up，人工审查认为观感廉价，收敛为首页一处入场编排 + 内容区按需揭示。
- 收藏按钮不可见：AI 在深色背景上用灰色图标，人工审查发现点击后图标消失，改为常显朱砂色实底 + 状态切换保留图标。
- 打字机定时器泄漏：AI 实现流式打字机但未在视图切换时停掉定时器，人工审查发现跨视图残留，统一在视图切换时停掉隐藏视图常驻定时器。
- 窄屏图谱不可读：AI 为适配移动端把 1000px 图谱压成 390px 导致节点字被压缩，人工审查回退并固化"窄屏最小 1000px + 横滑"规则。

- **3D 模型粗糙**：AI 程序化几何再精也缺真实质感，人工审查后加入实物影像模态，一键切换真实文物影像与 3D 模型对照。

验证方式不依赖"感觉"：断网实测、双端截图、console 零报错、滚轮帧率观察、node --check 语法校验。每轮 verify 字段记录在 evolution-data.js 中，评委可对照复核。

第二章　架构设计

——双轨产物、离线优先、双层 AI

2.1　模块化源码 vs 单文件编译双轨

开发态是 14 个 JS 文件 + HTML 结构，便于阅读调试；交付态是 `bundle_single.py` 内联成一个单文件 HTML，双击即运行。源码与产物分离，评委 clone 后跑一条命令得到可演示产物。

开发态（模块化源码）

```
index.html          # 结构 + 设计系统 + 质感动效层
app.js              # 应用/交互层：10 视图渲染、路由、问答逻辑
data.js             # 核心数据层：25 项非遗 + 离线知识库 + 图谱连边
map-data.js         # 河南 SVG 轮廓 + 地市坐标
regions-data.js     # 十八城介绍 + 地标
treasury-data.js    # 藏库文字层：15 件藏品
treasury-real.js    # 藏库影像层：实物图 base64
treasury-audio.js   # 藏库讲解音频：10 段 WAV base64
treasury-process.js # 工艺流程：5 件分步工序
treasury.js         # 藏库视图层：展柜、讲解播放器
three-showcase.js   # WebGL 3D：汝瓷碗 + 榫卯拆解
evolution-data.js   # 演进记录：12 轮迭代
help-data.js        # 图文使用说明
vendor/             # three.js r186 + OrbitControls + Lenis（本地化）
```

交付态（单文件编译）

一条命令产出：

```
py -3 scripts/bundle_single.py
# 输出：dist/非遗智汇·多彩面板版.html（39.01 MB）
```

产物将全部 JS、字体 CSS、图片 base64、音频 base64 内联到一个 HTML 文件中，双击用浏览器打开即可运行，无需任何本地服务器、无网络请求。

双轨设计的好处：开发时可以按模块阅读、调试、修改；交付时一条命令编译成评委可直接打开的单文件。源码改动后重新运行脚本即可保持产物与源码同步。

2.2　数据/视图严格解耦

数据层纯数据，视图层纯逻辑，结构与样式分离。数据扩展时视图自动适配，不需要逐处改硬编码。

分层结构

层	文件	职责	不做什么
数据层	data.js / map-data.js / regions-data.js / treasury-*.js	25 项非遗字段、地市坐标、藏库文字/影像/音频、工艺流程	不操作 DOM、不写事件监听
视图层	app.js / treasury.js / three-showcase.js	10 视图渲染、路由、问答匹配、图谱/地图/海报逻辑、WebGL 场景	不写业务数据、不硬编码字段
结构层	index.html	HTML 结构 + CSS 变量 + 动效层	不写业务逻辑

数据扩展实例

项目从 15 项非遗扩充到 25 项时，只需要在 `data.js` 的 `FEIYI_DATA` 数组里追加新条目，视图层的统计函数 `ichStats()` 自动计算"25 项 / 6 门类 / 13 地市"，首页数据条、图谱节点、地图标点、收藏计数全部自动更新，无需逐处修改。

新增 5 项非遗时，同步在 `HENAN_CITIES` 补城市坐标、在 `GRAPH_KIN` 补连边、在 `ITEM_THEME` 补主题色——这些都是数据层工作，视图层零改动。

2.3 离线优先：零网络依赖

项目的核心硬约束是"断网可跑"。所有外部依赖在构建阶段内联，运行时不向任何第三方域名发请求。

字体子集化

原始 Noto Serif SC / Noto Sans SC 全套字体约 16MB，用 fonttools/pyftsubset 按页面实际用到的汉字与符号子集化，输出 woff2 并 base64 内嵌，最终字体 CSS 约 1,475 KB。加 `font-display: swap` 和系统字体回退栈（`SimSun`、`Microsoft YaHei`），避免白屏。

第三方库本地化

库	版本	本地路径	用途
three.js	r186	vendor/three.min.js	WebGL 3D 展柜（汝瓷碗 + 榫卯拆解）
OrbitControls	r186 同版	vendor/OrbitControls.min.js	轨道旋转/缩放/平移
Lenis	1.1.18	vendor/lenis.min.js	平滑滚动（可选，失败降级原生滚动）

图片与音频内联

56 张入库图片全部转 base64 data URI 内联到 HTML，10 段讲解 WAV 音频（约 17MB）base64 内联到 treasury-audio.js。运行时零外部请求，断网功能完整。

断网验证方法

1. 打开 dist/ 单文件 HTML；
2. DevTools → Network → 勾选 "Offline"；
3. 刷新页面，应看到：首页水墨长卷、25 项展品卡片、地图标点、知识图谱、问答可答、3D 场景可渲染、音频可播放；
4. console 应零报错（Lenis 如不可用会静默降级，不影响功能）。

2.4 双层 AI：离线知识库 ⇄ 预留大模型接口

问·AI 模块采用双层架构：默认离线知识库作答，预留大模型接口填 key 即升级。诚实声明，不把关键词匹配冒充大模型。

离线层（默认，永不掉线）

`data.js` 中 `KB_ALL` 内置 25 项非遗的问答对，采用关键词打分匹配 + 多轮指代追问。用户问"它的产地在哪"，系统自动回指上一主题。纯本地作答，断网演示不断线。

在线层（可选，填 key 即用）

`app.js` 顶部 `REMOTE_API` 预留兼容 OpenAI Chat Completions 协议的接口：

```
var REMOTE_API = {
  enabled: false,   // 开关：true 才走远程大模型
  url: "",          // 接口地址
  key: "",          // API Key
  model: ""         // 模型 / 推理接入点 ID
};
```

三字段填齐后改为 `enabled: true`，回答即切换为大模型生成。不填时界面不出现"离线 ⇄ 在线"切换按钮，纯本地作答。

> 诚实声明：默认离线回答由内置知识库检索匹配产出，并非大模型生成；填 key 后才接入真·大模型。
> README 与界面均明确标注"离线知识库"，不夸大 AI 能力。

第三章　可复现性论证

——为什么不是一次性屎山，如何一键复现、非伪造

3.1　一键复现：幂等构建脚本

评委 clone 仓库后，一条命令产出可演示单文件：

```
git clone https://github.com/linchenan666/feiyi-zhihui.git
cd feiyi-zhihui
py -3 scripts/bundle_single.py
# 双击 dist/非遗智汇·多彩面板版.html
```

构建脚本六步流水线

步骤	动作	失败即停条件
1/6	前置文件检查：确认 index.html + 14 个 JS 文件存在	任一文件缺失，立即退出
2/6	key 完整性校验：确认 treasury-real.js 15 件实物图 + treasury-audio.js 10 段音频全部对齐	缺 key 立即退出
3/6	扫描 assets/video/ 登记 HALL_VIDEOS（可选，无视频则跳过）	视频目录不存在不报错
4/6	内联 14 个本地 JS 文件到 HTML	任一 JS 标签未匹配，立即退出
5a/6	内联子集化字体 CSS	字体文件缺失立即退出
5b/6	内联 assets/ 图片为 base64 data URI	引用不存在的图片立即退出；替换后校验无图片相对路径残留
6/6	写出产物到 dist/	—

幂等性保证

同一份源码重复运行脚本，产物字节一致（不写时间戳、不写随机数）。可通过 SHA256 验证：连续跑两次，产物哈希应相同。

脚本只用 Python 标准库（io、os、re、sys、base64、urllib），不需要 pip 安装任何依赖。评委环境只需 Python 3.8+。

3.2　细粒度 Git 历史

截至 2026-10-26，仓库共 100 次细粒度提交，首末提交跨 2026-09-09 至 2026-10-26 约 7 周。不是一次性 initial commit 堆完全部功能。

提交时间分布

日期	提交数	阶段
2026-09-09	7	项目初始化：数据层 + 骨架
2026-09-10	5	首页视觉 + 总览卡片
2026-09-11	39	核心功能密集开发：地图、图谱、问答、文创、沉浸
2026-09-12	11	交互打磨 + 3D 珍品馆
2026-09-13	5	性能优化 + 单文件编译
2026-10-05	6	数据真实性修正 + 图片版权合规
2026-10-13	7	字体子集化 + three.js 本地化
2026-10-14	1	AI 表述诚实化
2026-10-15	7	README 补全 + 目录结构
2026-10-16	2	数据扩充第一批
2026-10-17	4	数据扩充第二批 + 统计动态化
2026-10-18	5	赛事名统一 + 技术文档
2026-10-26	1	最终提交

每次提交对应一次 AI 核心对话或一个功能模块。提交信息用 `feat:` / `fix:` / `chore:` / `docs:` 前缀分类，评委可逐条 `git log --oneline` 对照复核。

3.3 非伪造证据：Prompt 快照 + 演进记录

作品不是事后补的文档，而是全过程有可追溯的证据链。

三层证据

证据层	位置	内容
Git 提交历史	`git log`	100 次提交，每次对应一个功能模块或一次 AI 对话，时间分布跨 7 周
Prompt 链快照	`docs/Prompt历史.md`	各模块完整 Prompt 原文，评委可查 AI 当时收到了什么指令

演进记录	`evolution-data.js`	早期 12 轮迭代的 prompt → AI 建议 → 引导修复 → 验证，四步完整记录

评委可验证的复核路径

1. `git log --oneline`：查看 100 次提交，确认每次对应一个模块；
2. `git log --format='%ad %s' --date=short`：确认提交时间分布跨 7 周，不是最后一天挤出来的；
3. 打开 `docs/Prompt历史.md`：对照某轮 Prompt，查看 AI 当时收到了什么约束；
4. 打开演进馆页面（或 evolution-data.js）：查看每轮 AI 建议、人工修复、验证方式；
5. 运行 `py -3 scripts/bundle_single.py`：确认一条命令产出单文件，连续跑两次产物哈希一致。

3.4 交付前自检流程

每次改动后固定流程，不依赖"感觉"判断质量。

步骤	命令/动作	通过标准
语法校验	`node --check app.js`	exit code 0，无语法错误
重新打包	`py -3 scripts/bundle_single.py`	6 步全过，产物写入 dist/
断网实测	DevTools → Network → Offline → 刷新	功能完整，console 零外部请求
浏览器实测	打开 dist/ 单文件，逐页点击 10 视图	路由切换正常、动效流畅、3D 可渲染
Git 提交	`git add <具体文件>` + `git commit`	不使用 `git add .`，避免带入 dist/ 和临时目录

构建脚本内置失败即停：前置文件缺失、key 数量不对、图片路径未内联，任一条件不满足立即非零退出，不静默吞错。这保证了产物与源码的一致性——如果打包失败，说明源码有缺失，不会产出一个看似完整实则残缺的单文件。

> 工程结论：本项目通过模块化源码与单文件编译双轨解耦开发态与交付态；通过数据/视图分层实现 25 项非遗的零改动扩展；通过字体子集化、第三方库本地化、图片音频 base64 内联实现断网功能完整；通过 100 次细粒度提交、Prompt 快照、演进记录和幂等构建脚本证明作品可复现、非伪造。